

BLM1031 STRUCTURAL PROGRAMMING - HW1

1. Summary

You are required to design a system for a university that tracks student and course registrations. The system must have a **dynamic structure** that allows an unlimited number of courses to be added for each student, and all data updates must be performed directly through memory addresses.

2. Data Structures (Structs)

Define the following structures at the beginning of your code:

```
typedef struct {
    char courseName[50];
    char courseCode[10];
    int credit;
    int midtermGrade;
    int finalGrade;
    float average;
} Course;

typedef struct {
    char name[50];
    int studentNo;
    Course *courses;    // Pointer for dynamic Course array
    int courseCount;    // Number of current courses
    float overallAverage;
} Student;
```

3. Required Functions (Prototypes and Descriptions)

Implement the following 15 functions strictly according to the given prototypes, without using break or continue.

Basic Management Functions

- **void initStudent(Student *stu, char *name, int no);**
Initializes the student structure with default values and sets the courses pointer to NULL.
- **void addCourse(Student *stu, char *name, char *code, int credit);**
Expands the course array by 1 using realloc and adds the new course. Grades are initialized to 0.
- **void deleteCourse(Student *stu, char *code);**
Removes the course with the given code from the array, shifts remaining elements, and shrinks memory using realloc.
- **void freeSystem(Student *stu);**
Frees all dynamically allocated memory for the student to prevent memory leaks.

Grade and Calculation Operations

- **void enterGrades(Student *stu, char *code, int midterm, int final);**
Finds the relevant course, updates grades, and calculates the course average immediately (%40 midterm, %60 final).
- **void calculateOverallAverage(Student *stu);**
Updates the student's weighted overall average based on all course averages and credits.
- **void updateCourse(Student *stu, char *code, char *newName, int newCredit);**
Updates course name and credit, then recalculates the overall average. It is sufficient to update the course only for the specified student.

Searching and Filtering

- **int searchCourse(Student *stu, char *code);**
Searches for a course code and returns its index if found, otherwise returns -1.
- **void findHighestGradeCourse(Student stu);**
Prints the course with the highest average and its grade for the given student.
- **void listFailedCourses(Student stu);**
Lists all courses of the student with an average below 50 and marks them as "FAILED".

Reporting and Listing

- **void printReportCard(Student stu);**
Prints student information, all courses, grades, and overall average in table format.
- **void creditLoadReport(Student stu);**
Calculates and displays the total credit load of the student.
- **void sortCoursesAlphabetically(Student *stu);**
Sorts courses alphabetically (A to Z) by name and prints them.

Advanced Data Manipulation

- **void updateStudentInfo(Student *stu, char *newName, int newNo);**
Updates the student's basic identity information.
- **void copyCourses(Student *target, Student source);**
Dynamically copies all courses from one student to another (for exemptions, etc.).

4. Assignment Rules and Expectations

- **Memory Management:** NULL checks must be performed after every `malloc` and `realloc` operation.

- **Function Design:** No function should use `break`; loops must be controlled using logical conditions.
- **Parameter Passing:** Functions that modify data must use pointers (&), while read-only functions should use value passing.
- **Error Handling:** Proper error messages must be displayed when attempting to delete or update a non-existent course.
- **Global Variables:** Absolutely no global variables or fixed-size arrays are allowed.

SUBMISSION REQUIREMENTS:

- `studentNo_code.c` source file
- `StudentNo-Report.pdf` containing screenshots proving the code runs successfully

FINAL DEADLINE: 30 April 2026, 23:59

SUBMISSION PLATFORM: online.yildiz.edu.tr

Submissions via email or other methods will not be accepted. Late submissions will not be accepted.

Plagiarism or internet-sourced solutions (including AI) will be graded as 0.